The value of this sort of feedback is huge. You've just discovered a new story for a requirement that had been overlooked or initially misunderstood: "Send follow-up notification emails to ex-members whose membership has just lapsed."

In addition, you now have a clearer understanding of what's expected regarding the "Configure notification message texts" story, which we'll look at next. Originally this was conceived of as a configurable template that the development team could change when required and deliver in the next release. But based on this conversation, you now know that the marketing people want to be able to configure the message at any time. So you can now describe this next story as follows:

```
Story: Include a configurable incentive in the follow-up notifications
In order to increase retention rates for our Frequent Flyer program
As a sales manager
I want to be able to include a configurable text describing incentives to
rejoin such as discount offers or bonus points in the notification message
```

This example illustrates another point. User stories allow you to put off defining detailed requirements until as late as possible. As time goes on, you'll learn more and more about the system you're delivering. This is what the ongoing conversations promoted by BDD are designed to facilitate. If you specify the details of a user story too early, you may miss some important fact that you'll learn later on. If you went ahead and specified the details of the "Configure notification message texts" story first, you'd implement a piece of functionality with little business value, and it wouldn't correspond to the stakeholders' expectations at all.

But you can't procrastinate forever. If you leave it too late, you won't have time to talk to stakeholders and understand the detailed requirements before the feature is due. In lean software development, this is known as the last responsible moment. This concept is also heavily used in an approach called Real Options, which we'll look at in more detail in section 4.3.

EXERCISE 4.2 Develop the features you defined in the previous exercise, and break them down into stories of different sizes until you get to stories that you think are of a manageable size. Describe some of them in more detail using the "in order to ... as a ... I want" format.

4.1.4 A feature is not a user story

In many projects, the features we've been discussing would be represented as high-level user stories, and some teams don't find it necessary to break the features down into smaller stories. This is fine and will work well on smaller projects.

But there are some advantages to keeping a distinction between the two. Remember,

- A *feature* is a piece of functionality that you deliver to the end users or to other stakeholders to support a capability that they need in order to achieve their business goals.
- A *user story* is a planning tool that helps you flesh out the details of what you need to deliver for a particular feature.